DBZSCC

CODENAME 26-11

ARCHITECTURE & PRODUCT VISION

Lokale AI-native Entwicklungsplattform
Code Intelligence · MIRO · Agent Orchestration · Runtime · Memory · Control Center

KONSOLIDIERTE ZIELARCHITEKTUR | VERSION 1.0 | 02. NOVEMBER 2026

Entscheidungen, technische Baselines, Vorschläge und Implementierungsstatus klar getrennt.

LEITMOTIV — ANALYSIEREN. BELEGEN. PLANEN. FREIGEBEN. UMSETZEN. VERIFIZIEREN. LIEFERN.

Dieses Papier beschreibt das Zielbild von DBZSCC Codename 26-11. Es ist eine konsolidierte Architektur- und Produktvision, kein Nachweis dafür, dass alle beschriebenen Komponenten bereits implementiert oder getestet sind.

Inhaltsverzeichnis

01 Executive Summary
02 Produktidentität und Geltungsbereich
03 Systemarchitektur
04 Code Intelligence — der Produktkern
05 MIRO — Model Intelligence & Resource Orchestrator
06 Hardware Intelligence und Runtime Control
07 Benchmark Lab und Use-Case Model Profiles
08 Agentenarchitektur und Cognitive Orchestration
09 Hybrid Memory und Retrieval
10 UI/UX — Industrial Developer
11 Development Control Center und Project Observer
12 Datenhaltung, Events und Single Source of Truth
13 Sicherheit, Freigaben und Ausführungsgrenzen
14 Plattform- und Technologie-Baseline
15 Umsetzungsroadmap und Quality Gates
16 Verbindliche Architecture Guardrails
17 Statussprache und Evidenzstandard
18 Initialer Entscheidungskatalog
19 Produkt-Definition-of-Done
20 Quellenbasis, Konsolidierung und offene Validierung

01 | Executive Summary

DBZSCC Codename 26-11 ist als lokale, AI-native Entwicklungsplattform konzipiert. Der Produktkern ist nicht ein Chatfenster mit Zusatzfunktionen, sondern ein zusammenhängender Engineering-Workflow: Die Anwendung versteht ein Repository, belegt relevante Zusammenhänge, plant Änderungen, delegiert Aufgaben, führt freigegebene Aktionen aus und bindet Prüfergebnisse an konkrete Workspace- und Git-Zustände.

Produktversprechen: Aus einer Entwicklungsaufgabe wird ein nachvollziehbarer, kontrollierter und überprüfbarer Arbeitsprozess — von der ersten Analyse bis zum belegten Ergebnis.

Dimension	Zielbild
Produktbetrieb	Single-User-Produkt für den eigenen PC; local-first. Externe Dienste sind optionale Provider, nicht die Betriebsgrundlage.
Plattform	Cross-platform von Anfang an; plattformspezifische Fähigkeiten werden über Adapter und reale Kompatibilitätsprüfungen behandelt.
Agenten	Eigener DBZS Orchestrator besitzt globale Steuerung; LangGraph führt komplexe, zustandsbehaftete Workflows aus.
Modelle	Capability-basierte Model Registry und dynamische Ressourcensteuerung nach VRAM, RAM, CPU, Backend und laufender Last.
Memory	Modulares Hybrid-Memory mit Embeddings und Reranking; Workflow-Zustand bleibt vom langfristigen Projektwissen getrennt.
Kommunikation	Interne Services und Contracts; API nur für externe Integrationen, wenn benötigt.
Qualität	Diff-basierte Änderungen, Approval Gates, Tests und Evidence mit Commit-/Workspace-Bezug.

Bestätigte Architekturentscheidungen

- Hybrid-Orchestrierung: eigener DBZS Orchestrator plus LangGraph für komplexe Workflows.
- Dynamische Runtime: Modellstarts und Rollenbelegung werden ressourcenbewusst geplant.
- Interne API-Disziplin: Module kommunizieren über Services/Contracts; HTTP ist kein interner Zwang.
- Hybrid-Memory: Embeddings und Reranking sind Teil der Zielarchitektur.
- Cross-platform: Plattformneutralität wird durch Adapter hergestellt, nicht durch pauschale Gleichsetzung der Betriebssysteme.

Wichtige Abgrenzung

Die bereitgestellte UI- und MIRO-Vision definiert den Produktkontext von 26-11. Das Architektur-Blueprint ergänzt Code Intelligence, Runtime, Memory, Agenten, Sicherheit und Plattformstrategie. Das Development Control Center enthält einen konkreten, datierten Implementierungscheckpoint aus einem früheren Branch-Kontext. Dieser historische Status wird nicht als aktueller HEAD- oder Gesamtprojektstatus ausgegeben.

02 | Produktidentität und Geltungsbereich

Codename und Betriebsmodell

Produktname: DBZSCC
Codename: 26-11
Produktklasse: lokale AI-native Entwicklungs- und KI-Arbeitsumgebung.

- Primärer Nutzer ist eine einzelne Person auf dem eigenen PC.
- Lokale Modelle, lokale Dateien, lokale Runtime und lokale Ausführung bilden den Standardpfad.
- Hugging Face, GitHub, Gitea und Web-Recherche können als externe Integrationen genutzt werden.
- Cross-platform bedeutet portable Architektur und validierte Plattformadapter — nicht identische Runtime-Fähigkeiten auf jedem Betriebssystem.
- Multi-Tenant-, Cluster- und allgemeine Cloud-Plattform-Funktionen gehören nicht zum Produktziel.

Nicht-Ziele

- Keine Multi-Tenant-Verwaltung oder Benutzerorganisation.
- Keine Cluster-Orchestrierung oder verteilte Inferenzplattform als Grundvoraussetzung.
- Keine Cloud-first-Abhängigkeit für den Kernworkflow.
- Keine unbegrenzte Agentenautonomie und keine verdeckte Selbstmodifikation.
- Keine parallele Datenbank oder konkurrierende Task-Wahrheit ohne dokumentierte Migrationsentscheidung.

Local-first ist eine Produktentscheidung. Cross-platform ist eine Architekturentscheidung. Beides ist vereinbar, solange konkrete Fähigkeiten pro Plattform nachgewiesen werden.

03 | Systemarchitektur

Die Architektur folgt einer nach innen gerichteten Abhängigkeitsrichtung. Die UI stellt dar und delegiert. Die Anwendungsschicht koordiniert Use Cases. Der Domain Core definiert Verträge, Zustände und Regeln. Services kapseln fachliche Abläufe. Infrastruktur und Provider verbinden das System mit Datenbanken, Betriebssystemen, Modellen und Werkzeugen.

```
PRESENTATION
PySide6 / Qt · Views · ViewModels · Chat Workspace · Control Center
|
APPLICATION
Use Cases · DBZS Orchestrator · Task Lifecycle · Approval Policy
|
DOMAIN CORE
Contracts · Entities · Policies · Events · Statusmodelle
|
SERVICES
Code Intelligence · MIRO · Runtime · Memory · Git · Execution · Evidence
|
INFRASTRUCTURE / PROVIDERS
SQLite · Filesystem · Processes · llama.cpp · Embeddings · GitHub/Gitea
```

Schicht	Verantwortung	Nicht-Aufgabe
Presentation	Zustand anzeigen, Eingaben erfassen, Aktionen an Services delegieren.	Direktes SQL, Prozessstart oder komplexe Geschäftslogik.
Application	Use Cases koordinieren, Policies anwenden, Aufgaben und Freigaben steuern.	Provider-spezifische Implementierungen fest verdrahten.
Domain Core	Frameworkneutrale Verträge, Regeln, Ereignisse und Zustände definieren.	Qt-, FastAPI-, llama.cpp- oder OS-Details kennen.
Services	Fachliche Operationen und wiederverwendbare Anwendungsfälle bereitstellen.	Unkontrollierte Nebenwege an Contracts vorbei schaffen.
Infrastructure	Persistenz, Prozesse, Netzwerk und Dateisystem kapseln.	Fachliche Entscheidungen treffen.
Providers	Konkrete Modelle, Tools, APIs und Backends anbinden.	Globale Orchestrierung oder Datenhoheit übernehmen.

Verantwortungsgrenzen für neue Änderungen

- UI und app.py bleiben Präsentation, Interaktion, Wiring und Delegation.
- Deterministische Fachlogik gehört in Services.
- Blockierende und lang laufende Arbeit läuft in Workern außerhalb des UI-Threads.
- Agenten verwenden dieselben freigegebenen Services und Contracts wie andere Aufrufer.
- Automationen triggern bestehende Use Cases; sie duplizieren keine Geschäftsregeln.
- Persistence Adapter kapseln Datenzugriff; Events transportieren strukturierte, korrelierbare Zustandsänderungen.

04 | Code Intelligence — der Produktkern

Code Intelligence verbindet Repository Awareness, Syntax- und Semantikanalyse, Code Graph, Hybrid Retrieval, Planung, kontrolliertes Editieren und Verifikation. Das LLM formuliert Hypothesen und Lösungsvorschläge; deterministische Werkzeuge liefern überprüfbare Fakten.

```
REPOSITORY → INDEX / AST → SYMBOLS & DEPENDENCIES → CODE GRAPH → HYBRID RETRIEVAL → PLAN → PATCH → TEST / REVIEW
```

Intelligenzschicht	Funktion	Beispielfrage
Repository Awareness	Dateien, Sprachen, Frameworks, Entry Points, Build- und Testsysteme erkennen.	Wo startet die Anwendung?
Syntax & Semantik	AST, Symbole, Imports, Typen und Referenzen extrahieren.	Wo wird diese Funktion aufgerufen?
Code Graph	Abhängigkeiten und Auswirkungen modellieren.	Welche Module sind von der Änderung betroffen?
Hybrid Retrieval	Text-, Symbol- und semantische Suche verbinden.	Wo existiert eine ähnliche Implementierung?
Reasoning & Planning	Aufgabe zerlegen, Risiken und Änderungsumfang bestimmen.	Welche Dateien und Tests müssen angepasst werden?
Controlled Editing	Kleine, nachvollziehbare Diffs erzeugen.	Wie wird die Änderung reversibel umgesetzt?
Verification	Tests, Lint, Typprüfung und Diff prüfen.	Welche Akzeptanzkriterien sind belegt erfüllt?

Engineering Loop

1. Verstehen: Workspace, Branch, Commit und Dirty State erfassen; Projektstruktur und Einstiegspunkte erkennen.
2. Belegen: relevante Dateien, Symbole, Abhängigkeiten und Tests mit Pfad-/Zeilenbezug abrufen.
3. Planen: Ziel, Dateien, Risiken, Alternativen und Akzeptanzkriterien definieren.
4. Freigeben: erforderliche Approval Gates anwenden, bevor schreibende oder folgenreiche Aktionen starten.
5. Ändern: kleine Diffs erzeugen und bestehende Nutzeränderungen schützen.
6. Prüfen: passende Tests und Qualitätsprüfungen tatsächlich ausführen; Logs und Exit-Codes erfassen.
7. Liefern: Diff, betroffene Dateien, Prüfbefehle, Ergebnisse, Risiken und offene Punkte berichten.

Leitregel: Pfade, Symbolreferenzen, Testresultate, Exit-Codes und Git-Diffs müssen aus realen Werkzeugergebnissen stammen. Modellbehauptungen sind kein Ersatz für Evidence.

05 | MIRO — Model Intelligence & Resource Orchestrator

MIRO verwaltet Modelle, Artefakte, Fähigkeiten, Hardwareeignung, Benchmarks und ausführbare Use-Case-Profile. Der Nutzer soll primär Aufgabe oder Rolle wählen können, nicht manuell einen Modell-Dateipfad erraten müssen.

Model Discovery und Registry

- Lokale Modellwurzeln rekursiv und inkrementell scannen; Pfade konfigurierbar halten.
- Lokale llama.cpp-Toolchains, Executables, DLLs, Backends und Buildvarianten inventarisieren.
- Optionale Provider für Hugging Face, GitHub-Releases und technische Recherche anbinden.
- Kanonische Modellidentität, lokale Artefakte, Varianten und Hashes getrennt führen.
- Duplikate erkennen; Herkunft, Lizenz, Version und letzte Prüfung dokumentieren.

Capability Extraction

Kategorie	Zu erfassende Merkmale
Identität	Name, Hersteller, Architektur, Version, Modellfamilie, Hash.
Format / Größe	GGUF und andere Formate, Parameterzahl, Dateigröße, Sharding, Quantisierung.
Kontext	Context Window, RoPE, Tokenizer, Chat Template.
Modalität	Text, Bild, Audio, Video; Vision/OCR soweit unterstützt.
Fähigkeiten	Chat, Coding, Reasoning, Tool Calling, JSON/strukturierte Ausgabe.
Spezialrollen	Embeddings, Reranking, Draft, Translation, Classification, Vision.
Runtime	Backend, llama.cpp-Build, unterstützte Features und Konfiguration.
Ressourcen	RAM, VRAM, KV-Cache, Offloading und Parallelitätsgrenzen.
Herkunft	Lokaler Pfad, HF-Repo, GitHub-Quelle, Lizenz und Zeitstempel.

1. Deklarierte Fähigkeiten aus Model Cards und Metadaten extrahieren.
2. Architektur- und Runtime-Kompatibilität prüfen.
3. Tokenizer- und Template-Kompatibilität validieren.
4. Praktische Funktionstests für die jeweilige Fähigkeit ausführen.
5. Ergebnis mit Evidenz, Zeitstempel und Vertrauensgrad speichern.

Eine Model Card allein beweist nicht, dass eine Fähigkeit in der konkreten Runtime-Konfiguration funktioniert.

Multimodale Paarung

- Basismodell und kompatiblen Projektor (z. B. mmproj) erkennen.
- Architektur, Runtime und Versionen prüfen; Paarungsstatus explizit führen.
- Modell und Projektor als gemeinsame Ladeeinheit behandeln.
- Speicherbedarf gemeinsam kalkulieren und Vision-Funktionstest dokumentieren.
- Ungeklärte oder nur wahrscheinliche Paare nicht stillschweigend als kompatibel markieren.

06 | Hardware Intelligence und Runtime Control

Die lokale Hardware ist die Referenz. MIRO und Runtime Controller berücksichtigen reale Geräte, Treiber, Backend-Fähigkeiten, aktuelle Speicherbelegung und parallele Aufgaben.

Ressource	Zu prüfende Signale
CPU	Modell, Kerne, Instruktionsmerkmale, Threadbudget und aktuelle Last.
RAM	Gesamt und verfügbar, Modell-Offload, Cache, parallele Worker.
GPU	Gerät, Architektur, Backend, Auslastung, dedizierter VRAM.
Shared Memory	Verfügbarkeit und Bandbreite; nicht als kostenloser zusätzlicher VRAM rechnen.
I/O	Modellpfad, freier Speicher, Ladezeit und Durchsatz.
Runtime	Prozesse, Sessions, Ports, Health, Prioritäten und aktive Jobs.

Resource Estimation

- Modellgewichte und Quantisierung.
- Kontextlänge, KV-Cache, Batch und Parallelität.
- mmproj und weitere multimodale Puffer.
- Runtime-Overhead und bereits belegte Ressourcen.
- Sicherheitsreserve für Betriebssystem und andere Anwendungen.

Schätzungen müssen Unsicherheit und Reserve ausweisen. Nach dem Start werden Prognose und tatsächlicher Verbrauch verglichen. Eine VRAM-Zahl allein ist keine Startgarantie.

Runtime Control Plane

```
TASK REQUEST → MODEL RESOLVER → RESOURCE ESTIMATOR → SCHEDULER → PROCESS MANAGER → BACKEND ADAPTER
```

- Modelle nach Capability, Kompatibilität, Priorität und Ressourcen auswählen.
- GPU-, CPU- oder Hybridbetrieb anhand realer Backend-Unterstützung entscheiden.
- Vor Start Ressourcen schätzen und reservieren; nach Start tatsächliche Werte und Health prüfen.
- Idle-Unload und Cold-Swap konfigurierbar halten; aktive Aufgaben nicht überraschend unterbrechen.
- Prozess- und Portidentität eindeutig verwalten; fremde Prozesse nicht versehentlich beenden.
- Fallbacks nur bei nachgewiesener Unterstützung aktivieren.

07 | Benchmark Lab und Use-Case Model Profiles

Benchmarkdimensionen

- Ladezeit, Time to First Token, Prefill- und Decode-Tokens pro Sekunde.
- Latenz, Durchsatz, RAM-/VRAM-Verbrauch und Verhalten bei langen Kontexten.
- Coding, Reasoning, Tool Calling und strukturierte Ausgabe.
- Vision-/Multimodaltests, Stabilität, Fehlerquote und Abbrüche.
- Vergleich einzelner Modelle und Kombinationen unter reproduzierbaren Bedingungen.

Kombinations- und Paarungstests

- Planner A mit Coder B; Coder A mit Reviewer B.
- Parallelbetrieb und Multi-Agent Consensus mit unterschiedlichen Rollenbesetzungen.
- Embedding- und Reranker-Kombinationen.
- Draft-/Target-Paare nur bei erfüllten Tokenizer- und Kompatibilitätsbedingungen.

Profilvertrag

Profilfeld	Inhalt
Identität	Stabile ID, Version, Use Case und Beschreibung.
Besetzung	Modellreferenzen, Rollen, Aufgabenverteilung und lokale Artefakte.
Ausführung	Hardware-/Backend-Zuordnung, Kontext- und Parallelitätslimits.
Ressourcen	Budgets, Reserven, Ladeprioritäten und Warm-up.
Resilienz	Fallback-Modelle, Timeouts, Retry-Regeln und Ausfallverhalten.
Kompatibilität	mmproj-/Draft-Paarungen, Runtime-Anforderungen und Validierungsstatus.
Evidence	Benchmark-Run, Testkonfiguration, Modellversion, Hardware- und Runtime-Build.

Profil-Lifecycle

1. Hardwareinventar und Profilversion laden.
2. Modell- und Projektor-Artefakte auflösen.
3. Kompatibilität und Ressourcenbudget validieren.
4. Runtime-Konfiguration vorbereiten und benötigte Modelle laden.
5. Warm-up und Health Check durchführen; Startbereitschaft oder konkreten Fehler melden.
6. Im laufenden Betrieb kontrolliert umschalten; laufende Jobs berücksichtigen.
7. Bei Fehlern Rollback auf den letzten gültigen Zustand ermöglichen.

Profilwechsel müssen kontrolliert und möglichst transaktional erfolgen. Ein Zielprofil wird erst aktiviert, wenn Kompatibilität und Ressourcen geprüft sind.

Profilnamen wie Development Swarm, Research Swarm, Low Resource oder Multimodal Studio sind zunächst Vorschläge. Sie gelten erst nach konkreter Modellzuordnung und reproduzierbarem Benchmark als validierte Profile.

08 | Agentenarchitektur und Cognitive Orchestration

Der DBZS Orchestrator besitzt die globale Steuerung. LangGraph wird dort eingesetzt, wo komplexe, verzweigte und zustandsbehaftete Workflows davon profitieren.

DBZS Orchestrator	LangGraph
Intent Resolution und globale Task-Verwaltung	Graph-Ausführung und Zustandsübergänge
Modell-/Rollenauflösung und Ressourcenplanung	Verzweigungen, Schleifen und Workflow-Knoten
Agent Lifecycle und Priorisierung	Checkpointing und Wiederaufnahme
Globale Berechtigungen und Fehlerpolitik	Workflow-lokale Fehlerbehandlung
Ergebnisaggregation und Quality Gate	Human-in-the-loop innerhalb des Workflows

Startrollen

Rolle	Verantwortung
Coding Orchestrator	Aufgaben zerlegen, delegieren und Ergebnisfluss steuern.
Research / Planning	Quellen bewerten, Optionen und Umsetzungsplan erstellen.
Architect	Architekturfolgen, Verträge und Abhängigkeiten prüfen.
Coder / Subtask Worker	Begrenzte, klar definierte Implementierungsaufgaben ausführen.
Code Reviewer	Diff, Architektur, Tests, Sicherheit und Risiken prüfen.
Testing Agent	Prüfstrategie und Testausführung über freigegebene Services koordinieren.
Documentation Agent	Betroffene Projektunterlagen konsistent aktualisieren.
Vision Specialist	Bilder und Dokumente analysieren.
Embedding / Reranking	Memory-Index und Relevanzbewertung unterstützen.

Jeder Agent besitzt einen einheitlichen Vertrag, Capability-Profil, Tool-Berechtigungen und Ressourcenbudget. Rollen sind von konkreten Modellen entkoppelt. Agenten umgehen keine Sicherheits- oder Architekturregeln.

COT / ToT / Consensus in der Oberfläche

- Aufgabenanalyse und Anforderungserfassung sichtbar machen.
- Mehrere Lösungswege und deren Bewertung im Planner darstellen.
- Machbarkeit, Ressourcenbedarf, Risiken und erwartete Qualität vergleichen.
- Agentenbeiträge und Review-Ergebnisse zusammenführen.
- Entscheidungen mit Kriterien, Belegen und offenen Widersprüchen präsentieren.

Private interne Chain-of-Thought-Inhalte werden nicht ungefiltert offengelegt oder gespeichert. Die Anwendung zeigt nachvollziehbare Zusammenfassungen, Pläne, Alternativen, Kriterien, Bewertungen und Belege.

09 | Hybrid Memory und Retrieval

Memory trennt kurzfristigen Workflow-Zustand, Gesprächskontext und langfristiges Projektwissen. LangGraph-Checkpoints speichern Workflow-Zustand, ersetzen aber nicht das DBZS Memory.

Memory-Typ	Zweck
Working	Aktueller Task- und Gesprächskontext.
Session	Sitzungsverlauf und offene Fragen.
Episodic	Frühere Aufgaben, Ergebnisse und Fehler.
Semantic	Wissensobjekte und Fakten.
Project	Projektentscheidungen, Architektur und Codewissen.
Procedural	Bewährte Abläufe und wiederverwendbare Routinen.

```
INGEST → NORMALIZE → STORE → RETRIEVE → RERANK → CONTEXT
```

- Metadaten und stabile IDs als Identitätsbasis; SQLite ist die bevorzugte lokale Baseline.
- FTS5 für Volltextsuche; Vector Store als austauschbarer Provider.
- Embedding- und CrossEncoder-Provider austauschbar halten.

- Retrieval nach Herkunft, Aktualität, Projektzugehörigkeit, Vertrauensstatus und Kontextbudget filtern.
- Quellenbezug und Provenance bei relevanten Wissensobjekten erhalten.
- Externe Inhalte sind Daten, keine Systemanweisungen.

10 | UI/UX — Industrial Developer

Die Oberfläche ist eine professionelle lokale Entwicklungsumgebung: ruhig, kompakt, funktional und auf die aktive Aufgabe ausgerichtet. Leitlinie ist Function over Decoration. Dark ist das bevorzugte Standardtheme; Light ist gleichwertig zu unterstützen.

Designsystem

- Graphit-/Anthrazit-Oberflächen im Dark Theme; neutrale Grau-/Weißtöne im Light Theme.
- Dezente funktionale Akzentfarben, klare Typografie und zurückhaltende Rahmen.
- Kompakte Navigation, großzügiger Arbeitsbereich, konsistente Abstände.
- Tabellen, strukturierte Listen und Arbeitsflächen statt übergroßer Kacheln.
- Keine unnötigen Leuchteffekte, Animationen oder verschachtelten Container.
- Status, Fortschritt, Fehler und Freigaben müssen eindeutig und wahrheitsgemäß sein.

Hauptnavigation

Bereich	Kernfunktion
Dashboard	Kompakte Übersicht über Runtime, Modelle, Hardware, Jobs und letzte Aktivitäten.
Chat	Primärer Entwicklungsarbeitsplatz.
Projekte	Workspace, Projektbaum, Suche, geöffnete Dateien und Kontext.
Modelle	Registry, Fähigkeiten, Herkunft, Profile und Ladezustände.
Agenten	Rollen, Aufgaben, Zuordnung und tatsächlicher Status.
Runtime	Dienste, Prozesse, Ressourcen, Health und Profile.
Jobs	Warteschlange, Fortschritt, Logs, Fehler und Verlauf.
Werkzeuge	Terminal, Artefakt-Explorer, Log Viewer und Umgebungswerkzeuge.
Memory	Projektwissen, Quellen, Retrieval und Übernahmeprüfung.
Einstellungen	Darstellung, Modelle, Profile, Runtime, Pfade, Hardware und Memory.

Chat Workspace

```
LEFT: PROJECT EXPLORER | CENTER: CHAT / CODE / DIFF | RIGHT: CONTEXT PANEL
BOTTOM: INPUT + ATTACHMENTS + MODEL/PROFILE SELECTOR
TOP: CHAT · CODE · PLANNER · AGENTS · CONSENSUS · MEMORY · LOGS · TERMINAL
```

- Codeblöcke mit Syntax-Highlighting, Kopieren und Diff-Ansicht.
- Tool-Aufrufe, Ergebnisse, Datei-/Projektbezüge und Agentenaktivitäten.
- Ausführungs- und Testergebnisse mit Logs und Evidence.
- Relevante Modell-, Profil-, CPU-, RAM-, GPU-, VRAM- und Runtime-Informationen kontextabhängig anzeigen.
- Freigabe- und Bestätigungsschritte vor relevanten Datei- oder Systemänderungen.
- Lange Vorgänge asynchron ausführen und Status sichtbar halten.

Keine bestehende Funktion darf zugunsten der Optik entfernt werden. Die tatsächliche Umsetzung erfolgt anhand der vorhandenen Projektarchitektur, nicht durch einen unnötigen GUI-Neubau.

11 | Development Control Center und Project Observer

Das Development Control Center ist das operative Cockpit für den Idea-to-Product-Workflow. Der separat bewegliche Project Observer ist eine read-mostly Projektion desselben kanonischen Projektzustands. Beide dürfen keine konkurrierenden Tasklisten oder Wahrheiten führen.

Ein Projektgraph — mehrere Ansichten

- Project: Repository-Identität, Workspace-Pfad, Branch, Remotes und HEAD.
- Decision, Vision, Requirement, Task und Milestone: stabile IDs, Status, Abhängigkeiten und Akzeptanzkriterien.
- Artifact: Pfad, Typ, Version/Hash, Herkunft und Freshness.
- RepositorySnapshot und PullRequest: Provider, Branch, Commit, Status und Beobachtungszeit.
- ExecutionRun und TestEvidence: Befehl, ausführbare Datei, CWD, Umgebung, Exit-Code und Ergebnisartefakte.
- SyncOperation und Event: Preflight, beabsichtigte Aktion, Ergebnis, Event-ID und Correlation-ID.

Operator-Oberfläche

- Projekt-/Repository-Übersicht mit Workspace, Branch, HEAD, Dirty State und Remotes.
- Projektgedächtnis mit Entscheidungen, Annahmen und verknüpften Quellen.
- GitHub-/Gitea-/Local-Repository-Snapshots mit Zeitstempel und Providerstatus.
- PowerShell-/Terminal-Ausführung mit Allowlist, exakter ausführbarer Datei und CWD.
- Tests und Evidence an den exakten Commit und die Umgebung binden.
- Sync-Plan mit Divergenz, Konflikten und erwarteten Änderungen vor Pull/Push/Mirror.

Observer-Statusmodell

Status	Bedeutung
Planning	Vision, Anforderungen, Architektur und Meilensteine werden vorbereitet.
Todo	Freigegebene Arbeit ist startbereit.
Pending	Warten auf Entscheidung, Abhängigkeit, Provider oder externe Antwort.
In Progress	Arbeit mit dokumentiertem Owner oder Run läuft.
Blocked	Expliziter Blocker mit Grund.
Review	Ergebnis wartet auf Prüfung oder Freigabe.
Done	Akzeptanzkriterien mit Evidence erfüllt oder genehmigte Ausnahme dokumentiert.
Failed	Aktion oder Test ist fehlgeschlagen und muss untersucht werden.

Done ist kein Modellurteil. Ein Task wird erst Done, wenn Akzeptanzkriterien und relevante Evidence vorliegen oder eine dokumentierte manuelle Ausnahme genehmigt wurde.

12 | Datenhaltung, Events und Single Source of Truth

DBZSCC benötigt eine workspace-weite Single Source of Truth. Markdown bleibt ein wichtiges Export- und Präsentationsformat, aber keine konkurrierende kanonische Datenbank.

Grundregeln

- Vorhandene WorkspaceDB, Projekt-/Session-/Job-/Message-Persistenz und Eventpfade nach Schema-Review wiederverwenden.
- Keine zweite unabhängige Task- oder Statusdatenbank ohne dokumentierte Migrationsentscheidung.
- Stabile IDs, Zeitstempel, Provenance, Source Authority und Correlation IDs erhalten.
- Remote-Providerstatus als zeitgestempelte Beobachtung behandeln, nicht als kanonische Absicht.
- Konflikte und veraltete Daten sichtbar machen; nicht stillschweigend überschreiben.
- Generierte Dokumente als Projektionen kanonischer Zustände behandeln; Quelle und Generierungszeit dokumentieren.

Event- und Evidence-Modell

Events sind strukturierte, versionierbare Beobachtungen. UI-Benachrichtigungen ersetzen keine dauerhafte Evidence. Ausführungs- und Testergebnisse werden mit Task, Workspace, Revision, Umgebung, Befehl, Exitstatus und Logs verknüpft, soweit vorhanden.

Dokumentationsfrische

- README, Architektur, Projektstatus, TODO, Handover und Verifikation bewusst auswählen und im selben Review-Zyklus aktualisieren, wenn betroffen.
- Manuell verfasste Abschnitte erhalten; generierte Abschnitte klar abgrenzen und sicher ersetzbar halten.
- Drift melden, wenn Quellzustand oder Dokumentation auseinanderlaufen.
- Keine nutzerverfassten Inhalte stillschweigend überschreiben.

13 | Sicherheit, Freigaben und Ausführungsgrenzen

Level	Aktion	Kontrolle
L0	Lesen und analysieren	Automatisch, workspace-begrenzt.
L1	Vorschläge und Pläne	Automatisch, keine Seiteneffekte.
L2	Dateien ändern	Explizite Policy/Freigabe und reviewbarer Diff.
L3	Terminal und Prozesse	Kontrollierte Freigabe, Limits, exakte CWD/Executable.
L4	Commit / Push / Merge	Explizite Freigabe; Preflight-Diff; kein autonomes Merge.
L5	Systemweite Änderungen	Besonders deutliche explizite Freigabe.

Verbindliche Sicherheitsregeln

- Secrets ausschließlich über Environment Variables oder etablierte externe Credential-Mechanismen.
- Keine Secret-Werte in Projektzustand, Events, Logs, Dokumentation oder generierten Dateien.
- Workspace-Boundaries, Path-Traversal-Schutz und Eingabevalidierung.
- Command Injection und Prompt Injection als konkrete Bedrohungen behandeln.
- Agenten erhalten minimale Tool-Berechtigungen und umgehen keine Policies.

- Destruktive Aktionen, Installationen, Pushes, Merges und breite Systemeingriffe benötigen passende Freigaben.
- Retries möglichst idempotent gestalten; Event-/Operation-IDs gegen doppelte Arbeit verwenden.
- Ein isolierter Python-Subprozess ist keine belastbare Sandbox für nicht vertrauenswürdigen Code.

Ausführung und Synchronisierung

Lokale Ausführung, GitHub API, Gitea API und CI Runner sind getrennte Capability Provider. Remote UI-Zugriff bedeutet nicht automatisch Remote-Shell-Zugriff. Sync-Aktionen benötigen Vorabvergleich, sichtbaren Diff, Konfliktbehandlung und Ergebnisprotokoll.

14 | Plattform- und Technologie-Baseline

Bereich	Baseline / Kandidat	Statushinweis
Desktop UI	Python + PySide6 / Qt	Baseline aus Architekturunterlagen; konkrete Versionen validieren.
Inference	llama.cpp / GGUF	Primärer lokaler Kandidat; Builds/Backends inventarisieren.
Agenten	Eigener Orchestrator + LangGraph	Architekturentscheidung.
Persistenz	SQLite; SQLAlchemy/Alembic	Technologievorschlag; bestehende WorkspaceDB zuerst prüfen.
Contracts	Pydantic	Technologiebaseline aus Blueprint.
Async / HTTP	asyncio, HTTPX; FastAPI optional extern	Interne Services direkt; API nur für externe Integrationen.
Memory	FTS5 + austauschbarer Vector Store + Embedding/Reranking Provider	Konkreten Vector Provider validieren.
QA / CI	pytest, pytest-qt, Ruff, mypy optional, Gitea Actions	Prüfmatrix und reale CI-Umgebung validieren.
Plattformadapter	Windows, Linux, macOS	GPU-/Prozess-/Packaging-Verhalten je Plattform testen.

Die vorhandene requirements.txt nennt als aktuelle Paketanforderungen PySide6, requests, pydantic, psutil, pytest, pytest-qt und ruff. Diese Datei allein belegt weder die installierte Umgebung noch die vollständige Architektur oder erfolgreiche Tests.

Cross-platform-Strategie

- Gemeinsame Contracts für Process Manager, Hardware Detector, Filesystem, Paths und Environment.
- Windows: Prozessoptionen, Pfade, GPU Provider und Packaging.
- Linux: Prozesssignale, Pfade, GPU Provider und Packaging.
- macOS: Metal-/CPU-Pfade, Prozessmanagement und Packaging.
- Feature Detection statt pauschaler Kompatibilitätsannahmen.

15 | Umsetzungsroadmap und Quality Gates

Slice	Umfang	Abnahmeidee
01 Core Foundation	Contracts, Events, Config, Service Boundaries, Platform Interfaces.	Domain Core bleibt frei von UI-/OS-Abhängigkeiten.
02 Runtime & Scheduler	Process Registry, Resource Estimator, Scheduler, Backend Adapter.	Start/Stop/Health, Ressourcenprüfung und Fallbackpfad.

Slice	Umfang	Abnahmeidee
03 Model Registry	Scanner, Metadata, Capabilities, MM Pairing, Role Picker.	Modelle korrekt erkennen und nach Fähigkeiten filtern.
04 Orchestrator	Task Model, Agent Registry, Dispatcher, Aggregator.	Task wird nachvollziehbar zugewiesen und abgeschlossen.
05 LangGraph	Adapter, Graph Registry, Checkpointing, Approval Nodes.	Workflow pausiert und kann kontrolliert fortgesetzt werden.
06 Memory	FTS, Embeddings, Vector Provider, Reranking.	Relevante Treffer mit Herkunft und Projektfilter.
07 Execution & API	Permissions, Evidence, Sandbox, optionales API Gateway.	Nicht freigegebene Aktionen werden blockiert.
08 Cross-platform	Adapter, Packaging, CI Matrix, Regression.	Build-/Testnachweise je Zielplattform.

Quality Gates

- Ruff: Lint und Formatierung.
- mypy: Typprüfung dort, wo Typverträge etabliert sind.
- pytest: Unit- und Integrationstests.
- pytest-qt: GUI-Regressionen.
- Runtime Health Checks: Prozess, Backend, Modell und Ressourcen.
- Git-Diff Review: keine unbeabsichtigten Dateien oder Nutzeränderungen.
- CI: reproduzierbare Befehle und Ergebnisprotokolle.

Kein Big-Bang-Refactor. Jede Phase liefert ein kleines, prüfbares Inkrement. Bestehende Implementierungen werden wiederverwendet, wenn sie die Zielverträge erfüllen.

16 | Verbindliche Architecture Guardrails

Touched-area improvement

Wenn eine Änderung app.py oder ein anderes Modul berührt, ist die betroffene Verantwortung zu prüfen. Liegen dort Geschäftslogik, Persistenz, Orchestrierung, blockierende Arbeit oder provider-spezifische Integration an der falschen Stelle, soll sie im selben Schritt sicher extrahiert werden. Kein breiter Refactor nur aus Prinzip; riskante Extraktionen werden als konkreter Follow-up dokumentiert.

Pflichtcheck für jede relevante Änderung

1. Liegt die Verantwortung in der richtigen Schicht?
2. Lässt sich das fachliche Verhalten ohne vollständigen UI-Start testen?
3. Werden bestehende Services, Storage, Contracts und Events wiederverwendet?
4. Laufen blockierende Vorgänge außerhalb des UI-Threads?
5. Sind Fehler, Abbruch und Teilergebnisse explizit dargestellt?
6. Sind Zustandsänderungen und Ausführungsevidence nachvollziehbar?
7. Wurden Tests ergänzt/angepasst und tatsächlich ausgeführt, soweit möglich?
8. Unterscheidet die Dokumentation implementiertes Verhalten von Planung?
9. Ist das berührte Modul verständlicher geworden oder ist ein konkreter Follow-up erfasst?

Definition of Done

- Code geändert und Umfang benannt.
- Tests authored und tatsächlich ausgeführte Tests getrennt dokumentiert.

- Nicht ausgeführte Prüfungen und Gründe benannt.
- Bekannte Risiken und Follow-ups dokumentiert.
- Branch, Commit und PR-Status korrekt angegeben, sofern vorhanden.

17 | Statussprache und Evidenzstandard

Begriff	Verbindliche Bedeutung
Implementiert	Code ist vorhanden.
Getestet	Der benannte Test wurde tatsächlich ausgeführt; Ergebnis und Umfang sind angegeben.
Verifiziert	Definierte Akzeptanzprüfung wurde mit nachvollziehbarer Evidence bestanden.
Geplant	Beschlossen, aber noch nicht umgesetzt.
Vorgeschlagen	Option zur Entscheidung, noch nicht beschlossen.
Blockiert	Ein konkretes Hindernis verhindert den nächsten Schritt.
Unbekannt / nicht geprüft	Es liegt keine belastbare Evidence vor.

Ein Testergebnis gilt nur für den dokumentierten Commit, Befehl, die Umgebung und den Testumfang. Ein geänderter HEAD oder Dirty Working Tree kann frühere Evidenz entwerten. Keine Tests, Builds, Hardwaremessungen oder Live-Integrationen erfinden.

Mindestinhalt eines technischen Änderungsnachweises

1. Problem und beobachtbare Ausgangslage.
2. Umfang und bewusst ausgeschlossene Arbeiten.
3. Geänderte Dateien und Verhalten.
4. Risiken, Annahmen und bekannte Grenzen.
5. Tests/Prüfungen mit Befehl, Commit, Exitcode und Ergebnis — oder ausdrücklich nicht ausgeführt.
6. Aktualisierte Dokumente.
7. Verbleibende Punkte und nächster konkreter Schritt.

18 | Initialer Entscheidungskatalog

ADR	Entscheidung / Leitlinie	Einordnung
ADR-001	Domain Core bleibt unabhängig von UI- und Provider-Frameworks.	Architekturleitlinie
ADR-002	DBZS Orchestrator besitzt globale Task-Steuerung; LangGraph führt komplexe Workflows aus.	Bestätigt
ADR-003	Modellstarts laufen über zentralen Runtime Controller und Scheduler.	Zielarchitektur
ADR-004	Modelle, Capabilities, Rollen und Runtime-Profile sind getrennte Entitäten.	Zielarchitektur
ADR-005	Memory Provider sind austauschbar; SQLite bleibt Metadatenbasis.	Technologievorschlag
ADR-006	Interne Services sind direkt; externe API ist optional.	Bestätigt
ADR-007	Plattformabhängige Funktionen liegen in Adaptern.	Bestätigt
ADR-008	Codeänderungen sind diff-basiert, freigabegesteuert und Evidence-gebunden.	Architekturleitlinie
ADR-009	Control Center und Observer lesen dieselbe kanonische Projektzustandsbasis.	Produktentscheidung
ADR-010	Dokumentation ist Projektion und darf kanonische Zustände nicht duplizieren.	Verbindliche Policy

19 | Produkt-Definition-of-Done

Das Zielbild von DBZSCC Codename 26-11 ist erreicht, wenn die folgenden Fähigkeiten als durchgängiger, nachweisbarer Workflow zusammenarbeiten:

1. Ein Repository wird inventarisiert; Workspace-, Branch-, Commit- und Dirty-State-Identität sind bekannt.
2. Code-Suche kombiniert Text-, Symbol- und semantische Treffer.
3. Ein Task kann relevante Dateien, Symbole, Tests und Abhängigkeiten belegen.
4. Der Orchestrator delegiert passend zu Rolle, Modell, Berechtigung und Ressourcen.
5. Komplexe Workflows können pausieren, fortgesetzt und kontrolliert freigegeben werden.
6. MIRO erkennt lokale Modelle, Fähigkeiten, Runtime und Hardwareeignung mit nachvollziehbarer Evidence.
7. Einzelmodelle und Modellkombinationen können reproduzierbar getestet werden.
8. Use-Case-Profile lassen sich validieren, vor App-Start laden und zur Laufzeit kontrolliert wechseln.
9. Memory liefert projektbezogene Treffer mit Herkunft und Relevanzbewertung.
10. Änderungen werden als nachvollziehbare Diffs erzeugt; Nutzeränderungen bleiben geschützt.
11. Tests und Qualitätsprüfungen werden tatsächlich ausgeführt oder explizit als nicht ausgeführt markiert.
12. Development Control Center und Observer zeigen denselben Projektzustand und dieselbe Evidence.
13. Ergebnis, offene Risiken und Evidence werden in einem verständlichen Abschlussbericht geliefert.

DBZSCC 26-11 ist kein Chatfenster mit zusätzlichen Buttons. Es ist eine lokale Entwicklungsplattform, deren Intelligenz aus Codeverständnis, Modellkompetenzen, Ressourcensteuerung, Agentenkoordination, Memory, Werkzeugen und überprüfbarer Ausführung entsteht.

20 | Quellenbasis, Konsolidierung und offene Validierung

Diese Fassung wurde aus den im Projektkontext bereitgestellten Unterlagen konsolidiert:

- DBZSCC2611_MASTER.md: Universal AI Engineering Core, modulare Softwarearchitektur, AI-native Engineering, IDE-/Workspace-Funktionen, Sicherheit, Testing und Engineering Workflow.
- DBZSCC2611_MIRO_MODEL_INTELLIGENCE_VISION.md: Model Discovery, Capability Extraction, mmproj, Hardware Intelligence, Benchmark Lab, Profile Lifecycle und MIRO-Integration.
- DBZSCC2611_UI_VISION.md sowie GUI Strategie: Industrial Developer, Chat-zentrierter Workspace, kompakte Navigation, Dark/Light und Function over Decoration.
- DBZS_CODING_CHAT_Architecture_Code_Intelligence_Blueprint.pdf: bestätigte Architekturentscheidungen, Code Intelligence, Runtime, Agenten, Hybrid Memory, APIs, Plattformstrategie, Sicherheit und Roadmap.
- ARCHITECTURE_GUARDRAILS.md: Verantwortungstrennung, Single Source of Truth, sichere Ausführung und Definition of Done.
- DEVELOPMENT_CONTROL_CENTER.md: Projektgraph, Execution-/Test-Evidence, Observer, Sync und Dokumentationsfrische; Statusangaben bleiben an den dort genannten historischen Branch-/Zeitkontext gebunden.
- DOCUMENTATION_AND_PROJECT_PAPERS_POLICY.md: Pflichtprüfung, Dokumentationsaktualisierung, Statussprache und Evidence-Standard.
- requirements.txt: vorhandene Paketanforderungen als schmaler Konfigurationshinweis, nicht als Installations- oder Testnachweis.

Vor Implementierung gezielt zu validieren

- Exakte Repository-Struktur, aktueller Branch, HEAD und Working Tree.
- Vorhandene Services, WorkspaceDB-Schema, Event Bus, Job Spooler und bestehende Provider-Adapter.

- Konkrete Python-/Qt-Versionen und Windows/Linux/macOS-Kompatibilitätsmatrix.
- Verfügbare llama.cpp-Builds, GPU-Backends, Treiber und tatsächliche Modellpfade.
- Konkreter Vector Store, Embedding- und Reranking-Provider.
- Sandbox-Mechanismus je Betriebssystem.
- LangGraph Checkpointer, Prozess-/Runtime-Lifecycle und Packaging.

Dieses PDF ist ein überarbeitetes Zielbild und eine Architekturgrundlage. Es ist kein Repository-Audit, kein Implementierungsbericht und kein Testprotokoll.

DBZSCC | CODENAME 26-11

FUNCTION OVER DECORATION

ANALYSIEREN. BELEGEN. PLANEN. FREIGEBEN.
UMSETZEN. VERIFIZIEREN. LIEFERN.